

Sustentación Técnica: Modelo Multitenant en ChronoGest 2.1

Proyecto: ChronoGest 2.1 — Sistema de Gestión de Asistencia y Horarios del SENA

Arquitectura: NestJS + TypeORM + Angular + PostgreSQL + Redis + FastAPI

Modelo de multitenancy: Database-per-tenant (una base de datos física por inquilino)

Fecha: Junio de 2026

1. Resumen ejecutivo

ChronoGest 2.1 es una aplicación web diseñada para la gestión de asistencia, horarios y formación en el SENA. La solución fue construida bajo una arquitectura

multitenant, lo que permite que múltiples centros de formación (inquilinos o

tenants) utilicen una misma instancia de la aplicación, manteniendo sus datos completamente aislados en bases de datos físicas independientes.

Este documento describe el diseño técnico, los componentes involucrados, el flujo de una petición, las medidas de seguridad y las ventajas del modelo implementado.

2. Introducción

2.1 Contexto

El SENA cuenta con múltiples centros de formación a nivel regional y nacional. Cada centro gestiona sus propios aprendices, instructores, ambientes, fichas, horarios y registros de asistencia. Implementar un software independiente para cada centro sería costoso y difícil de mantener.

2.2 Solución propuesta

ChronoGest 2.1 resuelve este problema mediante una arquitectura

multitenant de base de datos separada (

database-per-tenant). Esto significa que:

- Todos los centros usan la misma aplicación web.
- Cada centro tiene su propia base de datos física en PostgreSQL.
- Los datos de un centro nunca se mezclan con los de otro.
- La aplicación selecciona dinámicamente la base de datos correspondiente en cada petición.

3. Objetivos

- 1 Permitir que múltiples centros de formación operen bajo una misma aplicación.
- 2 Garantizar el aislamiento total de los datos entre inquilinos.
- 3 Facilitar el mantenimiento centralizado y las actualizaciones del software.
- 4 Escalar horizontalmente la solución sin afectar a los inquilinos existentes.
- 5 Mantener la arquitectura hexagonal, sin modificar la lógica de dominio ni los casos de uso.

4. Marco teórico: Multitenancy

4.1 ¿Qué es un tenant?

Un

tenant (inquilino) es un cliente o entidad organizacional que comparte una misma aplicación con otros clientes, pero que opera sobre sus propios datos. En el contexto del SENA, cada centro de formación es un tenant.

4.2 Modelos de multitenancy

tenant_id

Modelo	Descripción	Ventajas	Desventajas
Base de datos compartida, esquema compartido	Todos los tenants en la misma tabla con una columna .	Fácil de mantener, poco costo.	Riesgo de fuga de datos, difícil de personalizar.
Base de datos compartida, esquema separado	Cada tenant tiene su propio esquema dentro de una misma BD.	Mejor aislamiento que el modelo compartido.	Más complejo de administrar.

Modelo	Descripción	Ventajas	Desventajas
Base de datos separada (database-per-tenant)	Cada tenant tiene su propia base de datos física.	Máximo aislamiento, cumplimiento normativo, fácil respaldo por centro.	Mayor uso de recursos, más conexiones.

4.3 Modelo seleccionado

ChronoGest 2.1 implementa el modelo

base de datos separada porque:

- Garantiza el máximo aislamiento de información sensible de aprendices e instructores.
- Facilita la recuperación de desastres y los respaldos individuales por centro.
- Permite personalizar esquemas si un centro requiere campos adicionales.
- Cumple con políticas de protección de datos personales.

5. Arquitectura del sistema

5.1 Stack tecnológico

Capa	Tecnología
Frontend	Angular 21
Backend API	NestJS 11
ORM	TypeORM 0.3.28
Base de datos	PostgreSQL 16
Caché y colas	Redis 7 + Bull
Reconocimiento facial	FastAPI + Python
Contenedores	Docker + Docker Compose

5.2 Diagrama de arquitectura general

tenantId de forma segura durante toda la vida útil de una petición asíncrona.

```
import { AsyncLocalStorage } from 'async_hooks'; export interface TenantContextStore { tenantId: string; } export const tenantStorage = new AsyncLocalStorage<TenantContextStore>(); export function getCurrentTenantId(): string | undefined { return tenantStorage.getStore()?.tenantId; } export function runWithTenant<T>(tenantId: string, callback: () => Promise<T>): Promise<T> { return tenantStorage.run({ tenantId }, callback); }
```

6.2 Middleware de extracción del tenant

Archivo:

`src/infrastructure/middleware/tenant.middleware.ts`

El middleware intercepta cada petición HTTP, lee el header

`x-tenant-id` y lo guarda en el contexto. Si no está presente, rechaza la petición.

```
import { Injectable, NestMiddleware, BadRequestException } from '@nestjs/common'; import { Request, Response, NextFunction } from 'express'; import { tenantStorage } from '../config/tenant-context'; @Injectable() export class TenantMiddleware implements NestMiddleware { use(req: Request, res: Response, next: NextFunction): void { const tenantId = req.headers['x-tenant-id'] as string; if (!tenantId) { throw new BadRequestException("Header 'x-tenant-id' es requerido"); } tenantStorage.run({ tenantId }, () => { next(); }); } }
```

Registro en AppModule:

```
export class AppModule implements NestModule { configure(consumer: MiddlewareConsumer) { consumer .apply(TenantMiddleware) .forRoutes({ path: '*', method: RequestMethod.ALL }); } }
```

6.3 Administrador de conexiones

Archivo:

`src/infrastructure/persistence/tenants/tenant-connection.manager.ts`

Gestiona un mapa en memoria de conexiones TypeORM (

`DataSource`) abiertas. Si la conexión para un tenant ya existe, la reutiliza; si no, la crea.

```
import { Injectable } from '@nestjs/common'; import { DataSource } from 'typeorm'; export interface
TenantCredentials { host: string; port: number; username: string; password: string; database:
string; } @Injectable() export class TenantConnectionManager { private tenantsDataSources = new
Map<string, DataSource>(); constructor(private readonly credentialsResolver:
TenantCredentialsResolver) {} async getTenantDataSource(tenantId: string): Promise<DataSource> {
const existing = this.tenantsDataSources.get(tenantId); if (existing?.isInitialized) { return
existing; } const credentials = await this.credentialsResolver.resolve(tenantId); const dataSource =
new DataSource({ type: 'postgres', host: credentials.host, port: credentials.port, username:
credentials.username, password: credentials.password, database: credentials.database, entities: [/*
entidades TypeORM del sistema */], synchronize: false, logging: process.env.NODE_ENV ===
'development', }); await dataSource.initialize(); this.tenantsDataSources.set(tenantId, dataSource);
return dataSource; } async closeAll(): Promise<void> { for (const [tenantId, dataSource] of
this.tenantsDataSources.entries()) { if (dataSource.isInitialized) { await dataSource.destroy(); }
this.tenantsDataSources.delete(tenantId); } } }
```

6.4 Proveedor dinámico de DataSource

Archivo:

`src/infrastructure/persistence/tenants/tenant-data-source.provider.ts`

El proveedor

`TENANT_DATA_SOURCE` tiene alcance por petición (

`Scope.REQUEST`). En cada petición, obtiene el

`tenantId` del contexto y retorna el

`DataSource` correspondiente.

```
import { Provider, Scope } from '@nestjs/common'; import { DataSource } from 'typeorm'; import {
getCurrentTenantId } from '../../config/tenant-context'; import { TenantConnectionManager } from
'./tenant-connection.manager'; export const TENANT_DATA_SOURCE = 'TENANT_DATA_SOURCE'; export const
tenantDataSourceProvider: Provider = { provide: TENANT_DATA_SOURCE, scope: Scope.REQUEST,
useFactory: async (connectionManager: TenantConnectionManager): Promise<DataSource> => { const
tenantId = getCurrentTenantId(); if (!tenantId) { throw new Error('No se encontró un tenantId en el
contexto actual'); } return connectionManager.getTenantDataSource(tenantId); }, inject:
[TenantConnectionManager], };
```

6.5 Ejemplo de repositorio de infraestructura

Archivo:

`src/infrastructure/persistence/repositories/typeorm-user.repository.ts`

Los repositorios no cambian su interfaz de dominio. Solo reciben el

`DataSource` dinámico mediante inyección.

```
import { Inject, Injectable } from '@nestjs/common'; import { DataSource } from 'typeorm'; import { IUserRepository } from '../domain/ports/user.repository.port'; import { User } from '../domain/entities/user.entity'; import { UserOrmEntity } from '../entities/user.orm-entity'; import { TENANT_DATA_SOURCE } from '../tenants/tenant-data-source.provider'; @Injectable() export class TypeormUserRepository implements IUserRepository { constructor( @Inject(TENANT_DATA_SOURCE) private readonly dataSource: DataSource, ) {} private get repository() { return this.dataSource.getRepository(UserOrmEntity); } async save(user: User): Promise<User> { const ormEntity = this.toOrmEntity(user); const saved = await this.repository.save(ormEntity); return this.toDomainEntity(saved); } async findOne(id: string): Promise<User | null> { const found = await this.repository.findOne({ where: { id } }); return found ? this.toDomainEntity(found) : null; } private toOrmEntity(user: User): UserOrmEntity { const entity = new UserOrmEntity(); entity.id = user.id; entity.email = user.email; entity.name = user.name; return entity; } private toDomainEntity(entity: UserOrmEntity): User { return new User(entity.id, entity.email, entity.name); } }
```

7. Flujo de una petición

1. El usuario abre la aplicación Angular. 2. Angular envía una petición HTTP con el header x-tenant-id. 3. TenantMiddleware extrae el tenantId y lo guarda en AsyncLocalStorage. 4. El controlador delega al caso de uso (capa de aplicación). 5. El caso de uso usa el repositorio de infraestructura. 6. El repositorio inyecta TENANT_DATA_SOURCE. 7. El proveedor obtiene el tenantId del contexto y pide el DataSource. 8. TenantConnectionManager devuelve o crea la conexión a la BD del tenant. 9. La consulta se ejecuta sobre la base de datos correcta.

8. Seguridad

8.1 Aislamiento de datos

Cada tenant posee su propia base de datos física. No existe una columna

`tenant_id` compartida que pueda filtrar datos entre centros.

8.2 Autenticación por request

El

`tenantId` viaja en el header

`x-tenant-id` y se valida en el middleware. Sin este header, la petición es rechazada.

8.3 Encriptación de contraseñas

Las contraseñas de los usuarios se almacenan con bcrypt (

\$2b\$10\$. . .). No se guardan en texto plano.

8.4 Respaldo y recuperación

Al tener bases de datos separadas, es posible realizar respaldos individuales por centro sin afectar a los demás inquilinos.

9. Ventajas del modelo implementado

Ventaja	Descripción
Aislamiento total	Cada centro tiene su propia base de datos física.
Escalabilidad	Se pueden agregar nuevos tenants sin modificar la aplicación.
Mantenimiento centralizado	Una sola versión del backend atiende a todos.
Cumplimiento normativo	Facilita auditorías y protección de datos personales.
Personalización	Es posible adaptar el esquema de un tenant sin afectar a otros.
Alta disponibilidad	Un problema en la BD de un tenant no afecta a los demás.

10. Caso de uso: Centros de formación del SENA

10.1 Escenario

Dos centros de formación utilizan ChronoGest 2.1:

- Centro de Formación Caldas
- Centro de Formación Palmira

10.2 Configuración

cf-caldas

tenant_cf_caldas_db

cf-palmira

Tenant ID	Base de datos	Centro
		Caldas
		Palmira

tenant_cf_palmira_db

10.3 Peticiones

```
GET /api/aprendices x-tenant-id: cf-caldas
```

El backend conecta a

tenant_cf_caldas_db y devuelve solo los aprendices de Caldas.

```
GET /api/aprendices x-tenant-id: cf-palmira
```

El backend conecta a

tenant_cf_palmira_db y devuelve solo los aprendices de Palmira.

11. Consideraciones de despliegue

11.1 Migraciones por tenant

Como cada tenant tiene su propia base de datos, las migraciones de TypeORM deben ejecutarse en cada una de ellas. Se recomienda un script que itere sobre el catálogo de tenants.

```
# Ejemplo conceptual for tenant in cf-caldas cf-palmira cf-buga; do npx typeorm migration:run -d  
"./config/data-source.$tenant.ts" done
```

11.2 Base de datos maestra

Se recomienda mantener una base de datos maestra (`master_db`) que almacene el catálogo de tenants y sus credenciales de conexión.

11.3 Conexiones y recursos

El

`TenantConnectionManager` mantiene un caché de conexiones abiertas. Se debe monitorear el límite de conexiones de PostgreSQL para evitar saturación.

12. Conclusiones

`AsyncLocalStorage`

- ChronoGest 2.1 implementa un modelo multitenant database-per-tenant robusto y seguro.
- La arquitectura hexagonal se respeta al 100 %: el cambio se aisló en infraestructura.
- El uso de ``, middleware y un administrador de conexiones dinámico permite seleccionar la base de datos correcta en cada petición sin modificar la lógica de negocio.`
- El modelo es ideal para el SENA, ya que permite centralizar el software y mantener el aislamiento de datos entre centros de formación.

13. Glosario

Término	Definición
Tenant	Inquilino o cliente organizacional que usa la aplicación.
Multitenancy	Arquitectura donde una aplicación atiende a múltiples tenants.
Database-per-tenant	Modelo donde cada tenant tiene su propia base de datos.
AsyncLocalStorage	API de Node.js para almacenar contexto asíncrono.

